

ИД31, вариант 42: Морской грузовой порт

Дискретно-событийная модель работы морского порта на языке C. Программа симулирует жизненный цикл судов: прибытие, швартовку, разгрузку кранами на склад и отбытие.

Выполнила

Бусыгина Ксения Сергеевна, БПИ253

Особенности реализации

- Очередь событий с сортировкой по времени.
- Ресурсы: буксиры, причалы, краны и склад с ограниченной вместимостью
- Причалы и краны имеют настройки совместимости с типами судов и грузов
- Если склад переполнен, то судно разгружает доступный остаток и уплывает

Структура проекта

```
homework-1/
├── Makefile                # Скрипт сборки и запуска
├── config.txt              # Конфигурация симуляции по умолчанию
├── run_tests.sh            # Скрипт автоматического тестирования
├── README.md              # Документация
├── include/                # Заголовочные файлы
│   ├── queue.h            # Очередь событий
│   ├── resources.h        # Структуры и API ресурсов порта
│   ├── ship.h             # Модель судна
│   ├── dispatcher.h       # Логика принятия решений диспетчером
│   ├── port.h             # Агрегатор всех сущностей порта
│   ├── simulation.h       # Главный цикл симуляции
│   ├── config.h           # Парсинг конфигурационного файла
│   ├── ui.h               # Отрисовка консольного дашборда
│   ├── logger.h           # Логирование
│   ├── statistics.h       # Сбор и вывод статистики
│   └── utils.h            # Базовые типы (состояния, события)
├── src/                    # Исходный код (реализация)
│   ├── main.c             # Точка входа, обработка сигналов (Ctrl+C)
│   ├── queue.c            # Реализация очереди событий
│   ├── resources.c        # Выделение и управление ресурсами
│   ├── ship.c             # Инициализация судов
│   ├── dispatcher.c       # Алгоритмы диспетчера
│   ├── port.c             # Управление состоянием порта
│   ├── simulation.c       # Движок симуляции
│   ├── config.c           # Чтение и парсинг config.txt
│   ├── ui.c               # Вывод состояния порта в консоль
│   ├── logger.c           # Запись событий в лог-файл
│   └── statistics.c       # Формирование итогового отчета
```

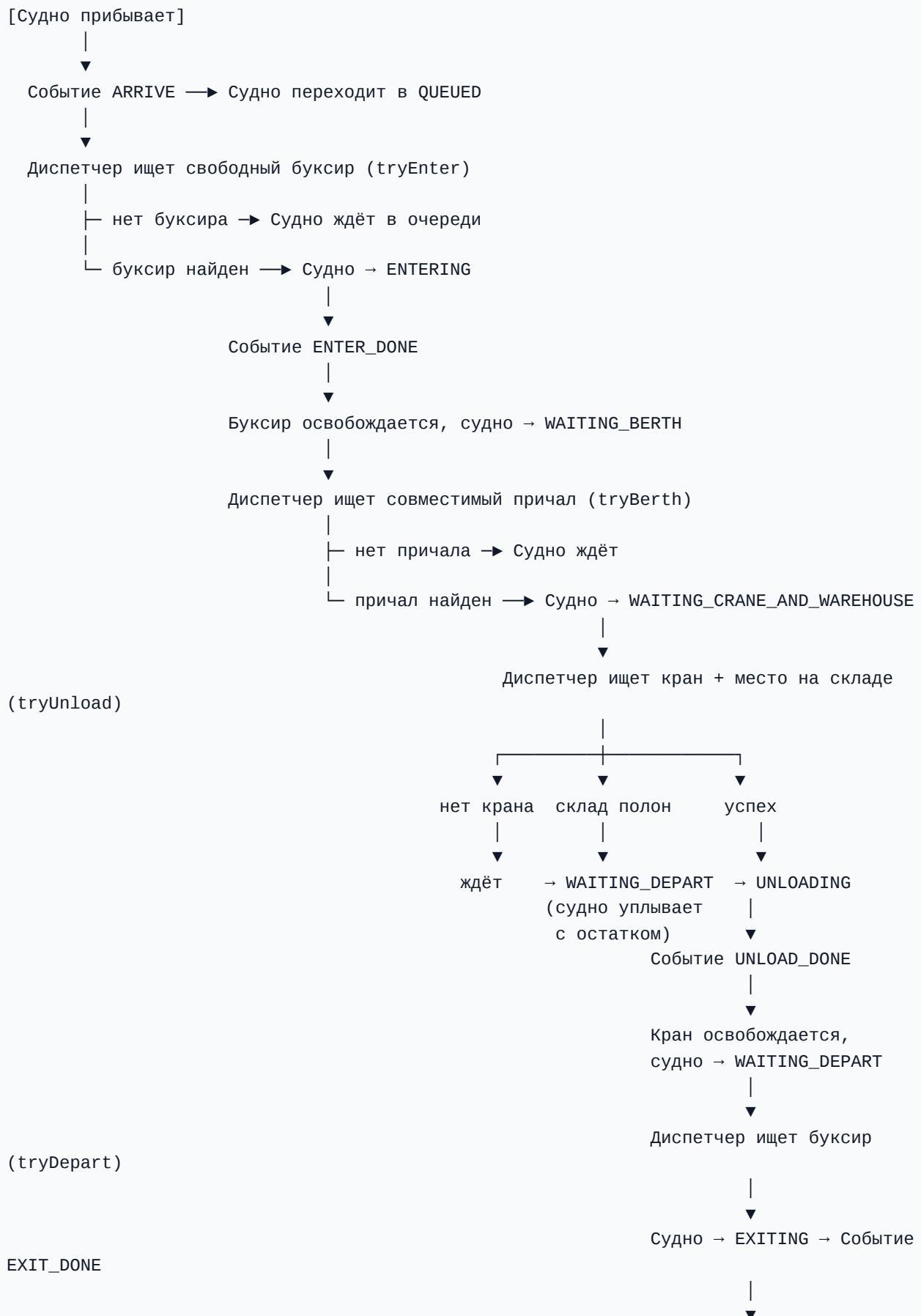
Сценарий работы

В основе симуляции лежит дискретная модель: время движется не непрерывно, а скачками от одного события к следующему. Все взаимодействия между сущностями описываются через события, которые хранятся в очереди (отсортированы по времени наступления).

Сущности и их роли

- **Ship** — проходит 9 состояний FSM: NOT_ARRIVED → QUEUED → ENTERING → WAITING_BERTH → WAITING_CRANE_AND_WAREHOUSE → UNLOADING → WAITING_DEPART → EXITING → DEPARTED
- **Tug** — ограниченный ресурс для входа/выхода судна
- **Berth** — ограниченный ресурс с совместимостью по типу судна (SMALL / MEDIUM / LARGE)
- **Crane** — ограниченный ресурс с совместимостью по типу груза (CONTAINER / LIQUID)
- **Warehouse** — ограниченный ресурс с фиксированной вместимостью
- **Dispatcher** — принимает решения о назначении ресурсов судам
- **EventQueue** — очередь событий, отсортированная по времени
- **Port** — агрегатор всех сущностей

Жизненный цикл судна



Буксир освобождается,
причал освобождается,
судно → DEPARTED

Ключевые особенности взаимодействия

1. Кран и место на складе резервируются одновременно. Если кран свободен, а склад полон, кран сразу освобождается, а судно отправляется на отбытие
2. Если на складе меньше места, чем груза на судне, разгружается только доступный остаток. Судно уплывает с частью груза
3. Случайные задержки: время входа/выхода и разгрузки варьируется в пределах 0-20% от базового значения (функция randomDelay)
4. Все случайные числа генерируются из одного значения параметра seed, указанного в конфиге. Одинаковый seed + одинаковый конфиг = идентичный результат

Требования

Для сборки, запуска и тестирования проекта вам понадобится следующее программное обеспечение:

Обязательные компоненты

- Операционная система: Linux, macOS или WSL (проект использует POSIX-совместимые системные вызовы (unistd.h, fcntl.h), поэтому Windows не поддерживается)
- Компилятор C: GCC версии 7.0 или выше с поддержкой стандарта C11/GNU11
- Система сборки: GNU Make
- Bash: Для запуска скрипта автоматического тестирования

Опциональные компоненты

- Doxygen: Для генерации HTML-документации по коду (см. раздел "Формирование документации")

Запуск

Клонирование проекта

```
git clone https://github.com/ksevlIII/homework-1.git
```

Переход в директорию

```
cd homework-1
```

Сборка

```
make clean && make
```

Запуск по умолчанию (с конфигом config.txt и задержкой 1000мс между тиками симуляции)

```
make run
```

Запуск с кастомными настройками

```
./port_sim <путь_к_конфигу> <задержка_в_мс>
```

Примеры:

- `./port_sim` — запуск с конфигом по умолчанию и задержкой 1000мс.
- `./port_sim config.txt 0` — запуск без задержки (мгновенное выполнение)
- `./port_sim my_config.txt 500` — запуск с кастомным конфигом и задержкой 0,5сек между тиками

Формат конфигурационного файла

Файл состоит из чисел, разделенных пробелами или переносами строк

```

max_time          # Максимальное время симуляции (тики)
tug_count         # Количество буксиров
berth_count       # Количество причалов
crane_count       # Количество кранов
warehouse_capacity # Вместимость склада (единицы груза)
enter_exit_time   # Базовое время входа/выхода судна
unload_time       # Базовое время разгрузки
ship_count        # Количество судов
seed              # Параметр для генератора случайных чисел

# Совместимость причалов (berth_count строк)
# Формат: berth_id small medium large (0 - не совместим, 1 - совместим)
0 1 1 0

# Совместимость кранов (crane_count строк)
# Формат: crane_id container liquid (0 - не совместим, 1 - совместим)
0 1 1

# Параметры судов (ship_count строк)
# Формат: type cargo_type volume arrival_time
# type: 0=SMALL, 1=MEDIUM, 2=LARGE
# cargo_type: 0=CONTAINER, 1=LIQUID
1 0 30 0

```

Пользовательский интерфейс

Программа предоставляет два канала вывода информации: консольный дашборд (в реальном времени) и лог-файл

Консольный дашборд

Дашборд перерисовывается на каждом тике симуляции и отображает текущее состояние порта в виде текстовой таблицы:

```

+----- ARKHANGELSK BAKARITSA PORT -----+
| Time: 5      / 25                               |
+----- RESOURCES -----+
| Tugs (3):   [ 0*] [ 1 ] [ 2 ]                 |
| Berths (2): [ 0#] [ 1 ]                       |
| Cranes (2): [ 0^] [ 1 ]                       |
| Warehouse: 30 / 100                           |
+----- SHIPS -----+
| #0 type=1 cargo=0 volume=30   | entering      |
| #1 type=2 cargo=1 volume=50   | not arrived  |
| #2 type=1 cargo=0 volume=40   | queued      |
| #3 type=0 cargo=1 volume=60   | not arrived  |
+-----+

```

Обозначения ресурсов:

- [0*] — буксир #0 занят (*), [0] — свободен
- [0#] — причал #0 занят судном (#), [0] — свободен
- [0^] — кран #0 работает (^), [0] — свободен
- Warehouse: 30 / 100 — использовано 30 из 100 единиц вместимости

Статусы судов:

- not arrived — судно ещё не прибыло (время прибытия не наступило)
- queued — судно прибыло и ждёт буксир для входа
- entering — судно заходит в порт с буксиром
- waiting berth — судно ждёт свободный причал
- waiting crane/warehouse — судно ждёт кран и место на складе
- unloading — идёт разгрузка
- waiting depart — судно разгружено, ждёт буксир для выхода
- exiting — судно выходит из порта
- departed — судно покинуло порт

Дашборд обновляется с задержкой, заданной вторым аргументом запуска (по умолчанию 1000 мс).
При запуске с задержкой 0 дашборд выводится мгновенно

Лог-файл port.log

Все действия симуляции записываются в файл port.log в хронологическом порядке:

```
=== Simulation Start ===
Resources: tugs=3, berths=2, cranes=2, warehouse=100
Seed: 100
[T=0000] Ship #0 arrived (type=1, cargo=0, vol=30)
[T=0000] Ship #0: tug #0 entering
[T=0001] Ship #1 arrived (type=2, cargo=1, vol=50)
...
[T=0012] Ship #3: tug #0 exiting

===== FINAL STATISTICS =====
Ships arrived:          4
Ships departed:         4
Ships pending:          0
Unloaded:               1
Total wait time:        10 ticks
Max queue length:       2
=====
=== Simulation Finished ===
```

Лог содержит:

- Параметры запуска (ресурсы, seed для генератора случайных чисел)
- Полную хронологию событий с метками времени [T=XXXX]
- Итоговую статистику: количество прибывших/отбывших судов, время ожидания, максимальную длину очереди, число неразгруженных судов

Завершение программы

Симуляция может завершиться по одному из трёх сценариев:

1. Штатное завершение (все суда обслужены) Программа завершается автоматически, когда выполнены оба условия:

- Все суда находятся в состоянии `DEPARTED` (успешно отбыли)
- Очередь событий пуста

В логе появляется запись:

```
=== Simulation Finished ===
```

Пример: при `maxTime = 25` и 4 судах, если все суда успевают пройти полный цикл до `T=25`, симуляция завершится досрочно (например, на `T=16`), не дожидаясь конца отведённого времени

2. Завершение по исчерпанию времени (`maxTime`)

Если к моменту `currentTime > maxTime` остаются суда в активных состояниях (`QUEUED`, `WAITING_BERTH`, `UNLOADING` и т.д.), симуляция принудительно завершается. Это моделирует ситуацию, когда порт не успевает обслужить все суда за отведённое время. В статистике такие суда будут учтены как `shipsPending` (остающиеся в порту)

Пример: если в конфиге `maxTime = 10`, а последнее судно прибывает на `T=8` и ему нужно 5 тиков на обслуживание, оно не успеет уплыть, и симуляция завершится на `T=10` с `shipsPending > 0`

3. Прерывание пользователем (`Ctrl+C`)

Пользователь может в любой момент прервать симуляцию комбинацией `Ctrl+C`. Программа обрабатывает сигнал `SIGINT` через обработчик `onSigint`, который устанавливает флаг `interrupted = true`. Главный цикл симуляции проверяет этот флаг на каждом тике и корректно завершает работу:

- Освобождаются все ресурсы (память, файловые дескрипторы)
- В лог записывается итоговая статистика на момент прерывания
- В лог добавляется пометка:

```
! Interrupted by user (Ctrl+C) !
```

Запуск тестов

Проверяет сборку, штатную работу, переполнение склада и воспроизводимость результатов

```
make test
```

Документация

Проект документирован с помощью Doxygen. Все публичные функции и структуры имеют комментарии в формате Doxygen.

Генерация документации

```
make doc
```

После выполнения этой команды в директории `docs/html` будет создан файл `index.html`. Откройте его в браузере для просмотра полной документации проекта.

Очистить директорию с документацией

```
make doc_clean
```